arXiv:2511.10376v5 [cs.CV] 17 Mar 2026

MSGNav: Unleashing the Power of Multi-modal 3D Scene Graph for Zero-Shot Embodied Navigation

Xun Huang[1,2,6] Shijia Zhao[1,2] Yunxiang Wang[4] Xin Lu[5,6]
Wanfa Zhang[1,2] Rongsheng Qu[3,6] Weixin Li[3,6]✉ Yunhong Wang[3,6] Chenglu Wen[1,2]✉
[1]Fujian Key Laboratory of Urban Intelligent Sensing and Computing, Xiamen University, China
[2]Key Laboratory of Multimedia Trusted Perception and Efficient Computing,
Ministry of Education of China, Xiamen University, China
[3]Beihang University, China [4]Nanyang Technological University, Singapore
[5]University of Chinese Academy of Sciences, China [6]Zhongguancun Academy, China

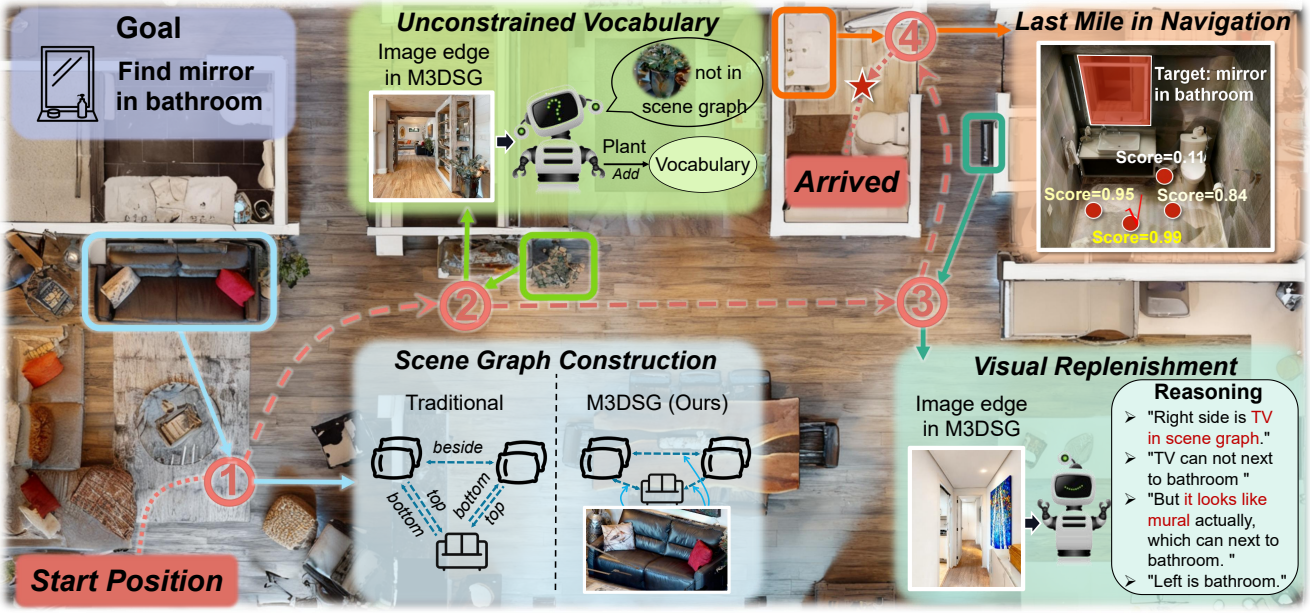


Figure 1. **One example illustrating the key insights of our work**. We introduce the Multi-modal 3D Scene Graph (M3DSG) as an alternative to traditional 3D scene graphs, enabling efficient scene graph generation. By incorporating dynamically preserved image-edge information, M3DSG supports unconstrained vocabulary and enhanced visual replenishment for the agent, thereby allowing more comprehensive and context-aware scene understanding in navigation tasks. Furthermore, to address the last-mile problem of selecting the optimal navigation viewpoint given a target location, we propose a visibility-based scoring mechanism for candidate viewpoints.

Abstract

Embodied navigation is a fundamental capability for robotic agents operating. Real-world deployment requires open vocabulary generalization and low training overhead, motivating zero-shot methods rather than task-specific RL training. However, existing zero-shot methods that build explicit 3D scene graphs often compress rich visual observations into text-only relations, leading to high construction cost, irreversible loss of visual evidence, and constrained vocabularies. To address these limitations, we introduce the Multi-modal 3D Scene Graph (M3DSG), which preserves visual cues by replacing textual relational edges with dynamically assigned images. Built on M3DSG, we propose `MSGNav`*, a zero-shot navigation system that includes a Key Subgraph Selection module for efficient reasoning, an Adaptive Vocabulary Update module for open vocabulary support, and a Closed-Loop Reasoning module for accurate exploration reasoning. Additionally, we further identify the "last mile" problem in zero-shot navigation — determining the feasible target location with a suitable final viewpoint, and propose a Visibility-based Viewpoint Decision module*

to explicitly resolve it. Comprehensive experimental results demonstrate that `MSGNav` *achieves state-of-the-art performance on the challenging GOAT-Bench and HM3D-ObjNav benchmark. The code will be publicly available at here.*

1. Introduction

Embodied navigation is a fundamental capability of robotic agents, enabling them to accomplish diverse tasks in realistic environments [47]. Such tasks are typically goal-oriented, with targets defined by objects, natural language descriptions, or images [45]. Recently, the challenging open-vocabulary embodied navigation task [52] has gained increasing attention, as real-world agents are required to generalize to a wide and unconstrained range of object categories [47].

Previous RL-based embodied navigation methods suffer from poor generalization and a large sim-to-real gap [44]. Inspired by rapid advances in pre-trained large model techniques [1, 17], zero-shot embodied navigation approaches have emerged at this historic moment. Zero-shot navigation system does not require any training or fine-tuning before being applied to real-world scenarios. These approaches [18, 43, 53] substantially reduce training time and computational cost while enabling open-vocabulary navigation, thereby facilitating deployment in real-world applications.

Notably, a series of methods [36, 37] that build explicit 3D scene graphs from observations and leverage LLMs to drive exploration have shown promising performance on standard goal-oriented navigation benchmarks [2, 41]. However, traditional 3D scene graphs overly abstract 3D object relations into simple textual labels (e.g., "top", "beside"), which seriously impedes progress on zero-shot embodied navigation. As illustrated in Fig. 1, the extreme abstracted 3D object-relation in traditional 3D scene graphs introduces three main limitations:

- ✘ **Expensive Construction**. Traditional 3D scene graphs rely on frequent MLLM queries for relation inference, incurring substantial token and time overhead.
- ✘ **Visual Deficiency**. Converting visual observations into text-only graphs discards visual evidence, leading to ambiguity and sensitivity against perception errors.
- ✘ **Constrained Vocabulary**. Novel categories beyond a preset vocabulary cannot be represented, limiting generalization in 3D scene graph-based methods.

Based on these observations, we conclude that visual information is indispensable to the real-world navigation process. Therefore, as illustrated in Fig. 1, we introduce the Multi-modal 3D Scene Graph (M3DSG), which replaces the pure-text relational edges with dynamically assigned images to incorporate visual cues, and facilitates efficient and open-vocabulary inference. Specifically, our M3DSG effectively addresses the three limitations above:

- ✓ **Efficient Construction**: M3DSG eliminates costly MLLM queries, greatly reducing construction overhead.
- ✓ **Visual Replenishment**: M3DSG provides visual contexts to enhance robustness against perception errors.
- ✓ **Unconstrained Vocabulary**: M3DSG utilizes preserved visual contexts to expand vocabulary, enabling open-vocabulary generalization dynamically.

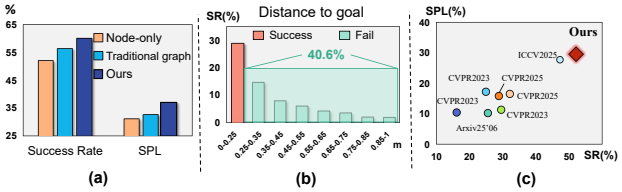


Figure 2. Performance comparisons between our `MSGNav` and other existing methods for embodied navigation on Goat-Bench [19]: the multi-modal open-vocabulary navigation benchmark. (a) The superiority of our M3DSG over traditional 3D scene graphs. (b) Distance statistics from the goal for the previous method (3D-Mem [43] as an example). (c) Our `MSGNav` system achieves state-of-the-art performance on the challenging Goat-Bench.

As shown in Fig. 2(a), our M3DSG markedly outperforms traditional text-only 3D scene graphs in the zero-shot embodied navigation task. Building on M3DSG, we further propose `MSGNav`, a navigation system that unleashes the power of **M**ulti-modal 3D **S**cene **G**raph for zero-shot embodied **Nav**igation. As illustrated in Fig. 1, `MSGNav` first selects target-relevant subgraphs from the complex multi-modal 3D scene graph by a key-subgraph selection module. During the exploration process, `MSGNav` updates the vocabulary using preserved visual information and incorporates a closed-loop reasoning module for accurate exploration reasoning, thereby enabling unconstrained vocabulary and reliable navigation.

Additionally, we realize that the *last-mile* problem in navigation tasks — knowing the target's location is not equivalent to determining a suitable navigation pose[1]. Existing zero-shot embodied navigation methods usually stop before that final step. As shown in Fig. 2(b), the existing 3D-Mem method [43] successfully approaches the target but still fails in many navigation tasks due to unacceptable viewpoints (distances ranging from 0.25 m to 1.0 m). To address this issue, we introduce a visibility-based viewpoint decision module in our `MSGNav`.

As shown in Fig. 2(c), `MSGNav` significantly outperforms state-of-the-art RL-based and zero-shot methods on the challenging embodied navigation GOAT-Bench [19], demonstrating consistent and substantial gains in zero-shot

✉ Corresponding author.

[1] *Cannot see the wood for the trees.* Being too close or occluded prevents a clear observation of the target, which is not a good viewpoint.

challenging embodied navigation. Our contributions can be summarized as follows:

- **M3DSG**: We propose a multi-modal 3D scene graph that incorporates visual information, overcoming pure-text limitations and enhancing open-vocabulary scene representation for embodied navigation.
- **Last-Mile**: We identify the last-mile problem and propose a visibility-based viewpoint decision module to address it.
- **MSGNav**: We develop a zero-shot embodied navigation system built on M3DSG, enabling active exploration and achieving state-of-the-art performance on the GOAT-Bench and HM3D-ObjNav embodied navigation datasets.

2. Related Work

2.1. Graph-based Scene Exploration

Unlike traditional feature representations [14–16], graph representations [9, 20] provide explicit spatial–semantic structures for scene reasoning and integrate well with LLMs/VLMs. Examples include SayPlan [35] for 3D scene graph planning, OVSG [3] for open-vocabulary grounding, imaginative world modeling [13], and BEV graph policies [30]. Graph reasoning has also been applied in navigation, e.g. SG-Nav [44] for hierarchical prompting and graph-retained adaptation [12] for cross-domain VLN. Yet, the overly abstract textual edges of current scene graphs limit the comprehensive scene context understanding, motivating our multi-modal 3D scene graph (M3DSG).

2.2. Zero-Shot Navigation

Zero-shot navigation allows agents to act in unseen environments without task-specific training, unlike supervised methods [33, 38] which require large-scale simulation. Existing zero-shot navigation methods span object-goal (ON), image-goal (IIN), and text-goal (TN). ON methods include CoW [7] with CLIP [32] frontier exploration, ESC [51], OpenFMNav [23], VLFM [46] using LLM reasoning, UniGoal [45] for multi-modal unification, and RATE-Nav [25] with region-aware termination. IIN methods range from Mod-IIN [22] to SIGN [42] for safety-aware drone navigation. TN methods such as InstructNav [28] generalize instruction following across ON, DDN, and VLN [4, 24, 29, 49]. However, most zero-shot work ignores the last-mile challenge of choosing an optimal viewpoint after target localization.

3. Approach

In this section, we formalize the zero-shot goal-oriented embodied navigation problem and provide an overview of our pipeline (Sec. 3.1). We then detail the construction of our Multi-modal 3D Scene Graph (M3DSG) (Sec. 3.2), and describe `MSGNav`, the navigation system that performs effective and closed-loop reasoning over this graph (Sec. 3.3).

3.1. Zero-shot Goal-Oriented Embodied Navigation

3.1.1. Problem definition

We consider a mobile embodied agent operating in an unknown or partially explored environment (*i.e.* the lifelong navigation setting). The agent receives target information in the form of an category, natural language description, or reference image. At each time step t, it obtains an RGB-D observation $\mathcal{I}_t$ and executes an action $\mathcal{A}_t$ (camera rotation or ego-motion) to actively explore until locating the target. The task is successful if the agent reaches any target viewpoint within d meters in at most T steps; otherwise, it fails. In the zero-shot navigation setting, the agent performs navigation without task-specific training or fine-tuning in simulation, thereby reducing training cost and improving generalization to unseen environments and targets.

3.1.2. Overview

Zero-shot goal-oriented embodied navigation remains highly challenging for conventional learning-based methods. Following current best practices, we utilize off-the-shelf pre-trained models: Vision Foundation Models (VFMs) and Vision Language Models (VLMs) for primitive reasoning. While these models excel in visual understanding, they are limited in 3D spatial reasoning. To address this gap, we propose the M3DSG, an explicit and multimodal 3D scene graph that is incrementally built to encode scene contexts while preserving the visual evidence. Based on M3DSG, the navigation system `MSGNav` completes goal-oriented navigation with a Visibility-based Viewpoint Decision (VVD) module.

Formally, at each time step t, M3DSG incrementally integrates the RGB-D observation $\mathcal{I}_t$ into an evolving scene graph $\mathbf{S}$. `MSGNav` then uses $\mathbf{S}$ as a prompt to guide VLMs in locating candidate target $\bar{o}$ positions or unexplored frontier for exploration. Finally, the VVD module selects an optimal navigation viewpoint $\mathbf{v}_{best}$ with the best line of sight to $\bar{o}$, enabling a successful final approach to the target.

3.2. M3DSG: Multi-modal 3D Scene Graph

Traditional 3D scene graphs represent object relations with text-only edges, which are overly abstract and hinder VLMs from accurately understanding scene contexts. Inspired by 3D-Mem [43], which emphasizes the value of raw images for navigation, we retain visual information when constructing an object-centric 3D scene graph. Unlike traditional 3D scene graph [9] which uses textual relation edges, our method stores images to describe detailed object relations directly. This image edge preserves the benefits of 3D scene graphs while avoiding repeated, costly model queries and providing a more holistic scene representation.

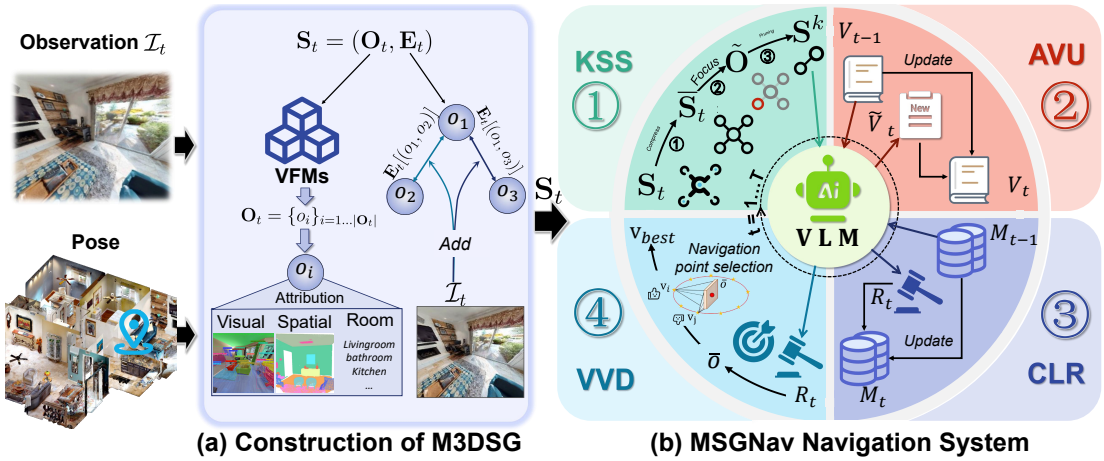


Figure 3. **The overall framework of our MSGNav**. At time step t, the agent incrementally constructs the scene graph S_t based on received observation $\mathcal{I}_t$ and its own pose. S_t includes a set of objects $\mathbf{O}_t$ with attributes, namely visual, spatial, and room properties, along with a set of image edges $\mathbf{E}_t$ representing object relationships. Subsequently, S_t is processed through KSS, AVU, and CLR modules, before being input to VLM query to obtain the target object $\bar{o}$. Finally, VVD module selects the insightful viewpoint $\mathbf{v}_{best}$ as a navigation point.

To mitigate the inference cost of numerous images during exploration, we design a dynamic allocation algorithm that efficiently converts the multimodal 3D scene graph into prompts, thereby requiring on average only **4 images** per query to represent the target-related scene context.

3.2.1. M3DSG Structure Definition

During the environment exploring process, M3DSG constructs a scene graph $\mathbf{S} = (\mathbf{O}, \mathbf{E})$, where $\mathbf{O} = \{o_i\}_{i=1}^{N_o}$ and $\mathbf{E} = \{\mathbf{e}_j\}_{j=1}^{N_e}$ denote the sets of objects and edges, respectively. Here, N_o is the number of detected objects, and $N_e = \sum_{1 \le x < y \le N_o} \mathbb{1}_{\|Pos_{o_x} - Pos_{o_y}\| < \theta}$ is the number of object pairs (o_x, o_y) whose spatial distance is below θ. Each object o_i is described by attributes including corresponding category, 3D coordinates, and room locations. Each edge $\mathbf{e}_j$ stores a set of RGB-D images $\mathbf{I}_j = \{\mathcal{I}_k\}_{k=1}^{|\mathbf{I}_j|}$ that represent the pair of corresponding objects to record the visual cues.

3.2.2. Incremental Construction of M3DSG

Following prior work [44, 45], the scene graph $\mathbf{S}$ is incrementally updated during exploration. At step t, $\mathbf{S}_{t-1}$ is updated to $\mathbf{S}_t$ with the current RGB-D observation $\mathcal{I}_t$ as:

$$\mathbf{S}_t = \mathcal{M}(\mathbf{S}_{t-1}, \mathcal{I}_t), \quad t \in [1, T], \quad \mathbf{S}_0 = (\varnothing, \varnothing), \tag{1}$$

where $\mathcal{M}$ is the graph update function consisting of Object $\mathbf{O}_t$ Update and Edge $\mathbf{E}_t$ Update.

Object Update. For the RGB-D observation $\mathcal{I}_t$ at time t, we extract the frame-level object set $\mathbf{O}_t^{frame} = \{o_i\}_{i=1}^{|\mathbf{O}_t^{frame}|}$ by VFMs: YOLO-W [5] for open-vocabulary detection, SAM [21] for instance masks, and CLIP [32] for visual embeddings. Each object o_i is represented as $o_i = \{\mathcal{ID}, \mathcal{C}, \mathcal{P}, \mathcal{B}, \mathcal{M}, \mathcal{PC}, \mathcal{V}, \mathcal{R}\}_i$, denoting unique ID, category, 3D position, bounding box, semantic mask, point cloud, visual feature, and room location, respectively. The global object set $\mathbf{O}_t$ is incrementally updated from the previous set $\mathbf{O}_{t-1}$ and the frame-level object set $\mathbf{O}_t^{frame}$ as:

$$\mathbf{O}_t = \Phi_{\text{merge}}\big(\Phi_{\text{match}}(\mathbf{O}_t^{frame}, \mathbf{O}_{t-1})\big) \cup \mathbf{O}_t^{frame}, \tag{2}$$

where Φ_{match} aligns objects in $\mathbf{O}_t^{frame}$ and $\mathbf{O}_{t-1}$ via spatial similarity $(\mathcal{B}, \mathcal{PC})$ and visual similarity $(\mathcal{C}, \mathcal{V})$, and Φ_{merge} merges features of matched objects [9]. This process maintains a global object set $\mathbf{O}_t$ within $\mathbf{S}_t$ step by step.

Edge Update. The core of an edge is a set of images $\mathbf{I}$ in which an object pair co-occurs, indexed by the unique IDs of the object pair, *i.e.* $\mathbf{E} : (\mathcal{ID}, \mathcal{ID}) \to \mathbf{I}$. Meanwhile, we maintain a mapping from each image to its corresponding object pair, *i.e.* $\mathbf{H} : \mathcal{I} \to (\mathcal{ID}, \mathcal{ID})$. For the current RGB-D frame $\mathcal{I}_t$ at step t with object set $\mathbf{O}_t^{frame}$ (from Object Update), we generate co-occurring object pairs (o_x, o_y) that appear together in $\mathcal{I}_t$ as:

$$\{(o_x, o_y) \in \mathbf{O}_t^{frame} \times \mathbf{O}_t^{frame} \mid o_x \neq o_y \ \wedge \ \|\mathcal{P}_{o_x} - \mathcal{P}_{o_y}\| \le \theta\}, \tag{3}$$

where θ is the adjacency distance threshold. The edge set $\mathbf{E}_t$ is then updated by retrieving the unique IDs $(\mathcal{ID}_{o_x}, \mathcal{ID}_{o_y})$ of each detected pair and appending $\mathcal{I}_t$ to the corresponding image set as:

$$\mathbf{E}_t[(\mathcal{ID}_{o_x}, \mathcal{ID}_{o_y})] \leftarrow \begin{cases} \{\mathcal{I}_t\} & \text{if } (\mathcal{ID}_{o_x}, \mathcal{ID}_{o_y}) \notin \text{dom}(\mathbf{E}_{t-1}) \\ \mathbf{E}_{t-1}[(\mathcal{ID}_{o_x}, \mathcal{ID}_{o_y})] \cup \{\mathcal{I}_t\} & \text{otherwise} \end{cases},$$

$$\mathbf{H}[\mathcal{I}_t] \leftarrow \begin{cases} \{(\mathcal{ID}_{o_x}, \mathcal{ID}_{o_y})\} & \text{if } \mathcal{I}_t \notin \text{dom}(\mathbf{H}) \\ \mathbf{H}[\mathcal{I}_t] \cup (\mathcal{ID}_{o_x}, \mathcal{ID}_{o_y}). & \text{otherwise} \end{cases},$$

$$\text{dom}(X) = \{k \mid \exists v : (k, v) \in X\}. \tag{4}$$

This edge update process is efficient, eliminating the need for costly VLM queries. Moreover, the stored image recorded in $\mathbf{E}_t$ provides compact and visually grounded

Algorithm 1 Greedy Dynamic Allocation for Pruning

Require: Image–Object Association Map: $\mathbf{H}$
Scene Graph: $\mathbf{S} = (\mathbf{O}, \mathbf{E})$, Related Objects: $\mathbf{O}^{rel}$

$\mathbf{E}^k \leftarrow \varnothing$ ▷ Key edges
$\mathbf{O}^k \leftarrow \mathbf{O}^{rel}$ ▷ Key objects
$\mathbf{U} \leftarrow \varnothing$ ▷ Uncovered relationship
$\mathbf{I}^{total} \leftarrow \varnothing$ ▷ Total images
for $o \in \mathbf{O}$ **do** ▷ Filter relevant edges
  if $\exists\, o^r \in \mathbf{O}^{rel}$: $(\mathcal{ID}_o, \mathcal{ID}_{o^r}) \in \mathrm{dom}(\mathbf{E})$ **then**
    $\mathbf{O}^k \leftarrow \mathbf{O}^k \cup o$
    $\mathbf{U} \leftarrow \mathbf{U} \cup (\mathcal{ID}_o, \mathcal{ID}_{o^r})$
    $\mathbf{I}^{total} \leftarrow \mathbf{I}^{total} \cup \mathbf{E}[(\mathcal{ID}_o, \mathcal{ID}_{o^r})]$
  end if
end for
while $\mathbf{U} \neq \varnothing$ **do** ▷ Greedy selection
  $\mathcal{I}^\star \leftarrow \arg\max_{\mathcal{I} \in \mathbf{I}^{total}} |\mathbf{H}[\mathcal{I}] \cap \mathbf{U}|$
  $\mathbf{E}^k[\mathbf{H}[\mathcal{I}^\star] \cap \mathbf{U}] \leftarrow \mathcal{I}^\star$ ▷ $\mathcal{I}^\star$ cover the most edges.
  $\mathbf{U} \leftarrow \mathbf{U} \setminus \mathbf{H}[\mathcal{I}^\star]$ ▷ Remove covered edges
end while
return $\mathbf{S}^k = (\mathbf{O}^k, \mathbf{E}^k)$ ▷ Key scene graph

representations of object relationships, which can be directly utilized for downstream exploration reasoning.

3.3. MSGNav Embodied Navigation System

M3DSG provides comprehensive scene representations that preserve rich visual information. To fully exploit this, we propose the navigation system MSGNav. As shown in Fig. 3(b), our MSGNav first significantly reduces the tokens and time cost required for inference by selecting key subgraphs (Sec. 3.3.1). MSGNav then addresses the out-of-vocabulary issue inherent to pre-set vocabulary by adaptively updating vocabulary with visual evidence from M3DSG (Sec. 3.3.2). Meanwhile, MSGNav performs closed-loop reasoning via decision memory and feedback (Sec. 3.3.3). Finally, MSGNav tackles the "*last-mile*" problem by optimizing navigation points through visibility-based viewpoint decisions (Sec. 3.3.4).

3.3.1. Key Subgraph Selection (KSS)

During exploration, the 3D scene graph $\mathbf{S}$ grows progressively and can become voluminous, thereby impairing the decision-making efficiency [43]. In embodied navigation, however, typically only a small related subgraph to the required target is meaningful. Therefore, processing the full scene graph consumes substantial computational resources without commensurate benefits. The key to addressing this issue is to preserve maximal relevant information with the minimal scene graph. MGSNav extracts the target-relevant subgraphs via a **Compress–Focus–Prune** procedure:

- **Compress**. We first simplify the rich but vast scene graph $\mathbf{S}$ into the compact adjacency list representation $\hat{\mathbf{S}} = (\hat{\mathbf{O}}, \hat{\mathbf{E}})$. For each object $\hat{o}_i \in \hat{\mathbf{O}}$, we retain only essential attributes (ID and category): $\hat{o}_i = \{\mathcal{ID}, \mathcal{C}\}_i$. The edge set is represented as undirected adjacency list: $\hat{\mathbf{E}}_{\hat{o}_i} = \{ID_{\hat{o}_j} | \hat{o}_j \in \hat{\mathbf{O}}, (ID_{\hat{o}_i}, ID_{\hat{o}_j}) \in dom(E)\}$.
- **Focus**. The compressed scene graph $\hat{\mathbf{S}}$ significantly reduces the overhead. Therefore, we directly fed it to the VLM to select top-k related objects $\mathbf{O}^{rel} \subset \mathbf{O}$ which are the most relevant to the current navigation target.
- **Pruning**. Finally, we construct the key subgraph $\mathbf{S}^k = (\mathbf{O}^k, \mathbf{E}^k)$ by progressively selecting the image that covers the most edges through a dynamic greedy allocation algorithm (in Algorithm 1).

The selected key subgraph $\mathbf{S}^k$ is used to query the VLM for exploratory reasoning with low cost (on average ∼4 images per query):

$$\mathcal{R}_t = \mathrm{VLM}(\mathbf{S}^k, \mathbf{F}, g, t), \tag{5}$$

where $\mathbf{F}$ denotes frontier images for exploration as in [43], g is the navigation target, and $\mathcal{R}_t$ denotes the selected target or frontier for task response. The specific prompt design is provided in the supplementary material.

This technique reduces the token cost by over 95% while feeding the information to VLM for reasoning, retaining most of the important objects relevant to the task. Consequently, the agent can efficiently focus on task-relevant entities, improving both navigation speed and performance.

3.3.2. Adaptive Vocabulary Update (AVU)

Existing methods achieve open-vocabulary navigation simply by using detectors, e.g., YOLO-W [5] and GroundingDINO [27], but remain constrained by preset vocabulary (e.g., ScanNet-200 [6], HM3D [41]). Such constraints hinder VLMs from handling diverse out-of-vocabulary scene representations in the real world. Additionally, preset vocabulary is difficult to accurately describe the image target.

To address this issue, MSGNav integrates adaptive vocabulary updates with preserved visual evidence during the exploratory reasoning process. We first initialize the vocabulary $V_{t=0}$ with the ScanNet-200. During exploration at time t, the VLM inspects edge images $\mathbf{E}_t$ and compares objects in $\mathbf{O}_t$ to propose required novel vocabulary $\hat{V}_t$ (based on the extension of Eq. 5):

$$\mathcal{R}_t, \hat{V}_t = \mathrm{VLM}(\mathbf{S}^k, \mathbf{F}, g, t). \tag{6}$$

Then the novel vocabulary $\hat{V}$ will be incorporated into V_t: $V_t = V_{t-1} \cup \hat{V}_t$. By continuously updating vocabulary V_t in this manner, the scene graph progressively captures richer and more accurate object descriptions, supporting navigation beyond preset vocabulary limitations.

3.3.3. Closed-Loop Reasoning (CLR)

Memory is critical for navigation agents. While prior work focuses mainly on scene memory, other forms of memory

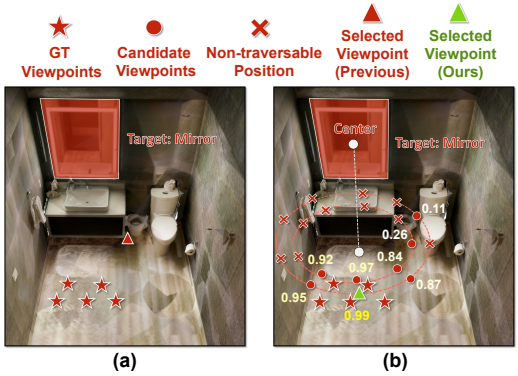


Figure 4. Demonstration of the "last-mile" problem. (a) Previous methods select the nearest traversable position after target localization, and often fail due to poor viewpoints. (b) Our VVD samples candidate viewpoints and computes visibility, which can select a suitable viewpoint close to GT for successful navigation.

can also influence decision-making. In addition to modeling the scene as perception memory, we introduce the decision memory $\mathbf{M}$ for closed-loop reasoning. At time step $t-1$, the exploration response $\mathcal{R}_{t-1}$ is stored in a historical action repository $\mathbf{M}_t$ as $\mathbf{M}_t = \mathbf{M}_{t-1} \cup \mathcal{R}_t, \mathbf{M}_0 = \varnothing$. Then $\mathbf{M}_t$ is utilized at time step t to assist subsequent decisions (based on the extension of Eq. 6) as:

$$\mathcal{R}_t, \hat{V}_t = \text{VLM}(\mathbf{S}^k, \mathbf{M}_t, \mathbf{F}, g, t). \tag{7}$$

This closed-loop reasoning enhances the accuracy of VLM strategies by memorizing past decision feedback to inform current decisions.

3.3.4. Visibility-based Viewpoint Decision (VVD)

For a navigation agent, the high-level decision process involves the location of the target, while the low-level planner converts it into executable actions. Therefore, an optimal final decision should choose a viewpoint with clear visibility to facilitate subsequent operations [39, 40]. Previous zero-shot methods [8, 18, 43, 44] often assume that knowing the target location is enough, and select the nearest traversable position as the navigation viewpoint. However, as shown in Fig. 4(a), this may yield viewpoints with poor visibility, causing failure even when the target $\bar{o}$ is correctly localized.

We term this the "*last-mile*" problem of navigation—identifying a suitable viewpoint despite correctly locating the target $\bar{o}$. To solve this problem, we realize that a good viewpoint should have clear visibility of the target. To achieve this goal, we propose a Visibility-based Viewpoint Decision (VVD) module (in Algorithm 2). As shown in Fig. 4(b), VVD uniformly samples candidate viewpoints around target $\bar{o}$ within radii $\mathbf{R} = \{r_j\}_{j=1}^{N_R}$. For each candidate $\mathbf{v}_i$, VVD computes a visibility score by evaluating occlusion between $\mathbf{v}_i$ and target point cloud $\mathcal{PC}_{\bar{o}}$. The viewpoint with the highest visibility score $\mathbf{v}_{best}$ is selected as the navigation coordinate. Compared with traditional choices, VVD yields viewpoints with superior target visibility.

Algorithm 2 Visibility-based Viewpoint Decision

Require: Target object $\bar{o}$, Scene point cloud $\mathcal{PC}$, Radii set $\mathbf{R}$, Samples number K, Camera height h, Obstruction distance τ

$\mathbf{c}_{\bar{o}} \leftarrow \frac{1}{|\mathcal{PC}_{\bar{o}}|} \sum_{\mathbf{p} \in \mathcal{PC}_{\bar{o}}} \mathbf{p}$ ▷ Target center
$\Theta \leftarrow \left\{ \frac{2\pi k}{K} \,\middle|\, k = 0, \ldots, K-1 \right\}$ ▷ Sampling angles
$\mathbf{V}_c \leftarrow \bigcup_{r_i \in \mathbf{R}} \{\mathbf{c}_{\bar{o}} + [r_i \cos\theta, h - \mathbf{c}_{\bar{o}}[1], r_i \sin\theta] \mid \theta \in \Theta\}$
$\mathbf{V}_c \leftarrow \{\mathbf{v} \in \mathbf{V}_c \mid \mathbf{v} \text{ is traversable}\}$ ▷ Candidate viewpoints
$S_{best} \leftarrow 0, \mathbf{v}_{best} \leftarrow \mathbf{0}$ ▷ Initialization
for each $\mathbf{v}_i \in \mathbf{V}_c$ **do**
  $\mathcal{Q}(\mathbf{v}_i, \mathbf{p}) = \left\{ \mathbf{v}_i + t \cdot \frac{\mathbf{p} - \mathbf{v}_i}{\|\mathbf{p} - \mathbf{v}_i\|} \,\middle|\, t \in [\tau, \|\mathbf{p} - \mathbf{v}_i\| - \tau] \right\}$
  $\mathcal{E}(\mathbf{v}_i, \mathbf{p}) = \forall \mathbf{q} \in \mathcal{Q}(\mathbf{v}_i, \mathbf{p}) : \min_{\mathbf{s} \in \mathcal{PC}} \|\mathbf{q} - \mathbf{s}\| \geq \tau$
  $S_{\mathbf{v}_i} \leftarrow \frac{1}{|\mathcal{PC}_{\bar{o}}|} \sum_{\mathbf{p} \in \mathcal{PC}_{\bar{o}}} \mathbb{1}_{\mathcal{E}(\mathbf{v}_i, \mathbf{p})}$ ▷ Visibility ratio
  if $S_{\mathbf{v}_i} > S_{best}$ **then**
    $S_{best} \leftarrow S_{\mathbf{v}_i}, \mathbf{v}_{best} \leftarrow \mathbf{v}_i$
  end if
end for
return $\mathbf{v}_{best}$ ▷ Best viewpoint maximizing visibility

4. Experiment

4.1. Experimental Setting

Datasets and Evaluation Metrics. We evaluate our proposed approach on two established goal-oriented navigation benchmarks: 1) *GOAT-Bench* [19] (Multi-modal lifelong open-vocabulary dataset, 360 episodes, 36 scenes, 2669 total subtasks, 36 novel goal categories). 2) *HM3D-ObjNav* (HM3D-Semantics-v0.2 [41] from 2023 Habitat Challenge, 1000 episodes, 36 scenes, 6 goal categories). To validate the generalization capability of our method to novel environments, all evaluations on GOAT-Bench are conducted on the *Val Unseen* split. Following standard practice, we assess navigation performance using Success Rate (SR $= \frac{N_{\text{success}}}{N_{\text{total}}}$) and Success weighted by Path Length (SPL $= \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} S_i \frac{l_i^s}{\max(l_i^s, l_i^a)}$), where $S_i \in \{0, 1\}$ denotes binary task success, and l_i^s and l_i^a represent the shortest path distance and the actual path traversed by the agent.

Implementation Details. We used the official success distance threshold from the benchmark: 0.25 for Goat-Bench and 1.0 for HM3D-ObjNav. We employ GPT-4o (2024-08-06) as our primary Vision-Language Model (VLM) via the OpenAI API. Additional analyses utilizing Qwen-VL-Max as an alternative VLM backbone, along with extended evaluations on the HM3D-OVON benchmark [47], are deferred to the supplementary material.

4.2. Main Experimental Results

We will show the main comparison results with other state-of-the-art methods on the Goat-Bench [19] and HM3D-

Method	Training-free	SR ↑	SPL ↑
SenseAct-NN Monolitic [19]	×	12.3	6.8
Modular CLIP on Wheels [19]	✓	16.1	10.4
Modular GOAT [19]	✓	24.9	17.2
SenseAct-NN Skill Chain [19]	×	29.5	11.3
VLMnav [8]	✓	20.1	9.6
DyNaVLM [18]	✓	25.5	10.2
3D-Mem† [43]	✓	28.8	15.8
TANGO [53]	✓	32.1	16.5
MTU3D [52]	×	47.2	27.7
MSGNav (Ours)	✓	**52.0**	**29.6**

Table 1. Experiments on the "*Val Unseen*" split of GOAT-Bench. "†" denotes the results we reproduced due to different settings.

Method	Training-free	SR ↑	SPL ↑
L3MVN [48]	✓	36.3	15.7
SG-Nav [44]	✓	49.6	25.5
InstructNav [28]	✓	58.0	20.9
CompassNav [26]	✓	59.6	26.9
Schrodinger'sNav [11]	✓	60.9	23.7
VLFM [46]	×	62.6	31.0
DORAEMON [10]	✓	66.5	20.6
WMNav [31]	✓	72.2	33.3
MSGNav (Ours)	✓	**74.1**	**33.4**

Table 2. Experiments on the HM3D-ObjNav benchmark.

ObjNav [41] benchmark.

4.2.1. Goat-Bench Benchmark

The results in Table 1 also show the outstanding performance of our MSGNav in tackling multimodal goals, lifelong navigation task settings. MSGNav achieves the best performance with 52.0% SPL and 29.6% SPL compared to both previously training-based and train-free methods. It surpasses the previous state-of-the-art training-based method MTU3D [52] by 4.8% SR and 2.1% SPL, demonstrating its superior navigation capabilities. These results highlight the effectiveness of our multi-modal scene graph in tackling multi-modal lifelong navigation tasks.

4.2.2. HM3D-ObjNav Benchmark

Table 1 demonstrates the superior zero-shot multi-modal open-vocabulary navigation capabilities of our MSGNav, we further evaluate it on the widely used HM3D-ObjNav benchmark. As shown in Table 2, MSGNav achieves a state-of-the-art Success Rate (SR) of 74.1%, which is 1.9% higher than that of the previous best-performing method WMNav [31], and significantly outperforms other prior methods. Although our Success Path Length (SPL) is nearly the same as WMNav without any significant advantage, this may be because the VVD module prioritizes viewpoints with a wide field of view over the absolute shortest path.

4.3. Ablation Analysis

In this section, we further analyze the effectiveness and advancement of each module in the MSGNav system. Specifically, this includes component ablation, the advantages of multimodal edges, and demonstrating how the VVD module aids in "*last-mile*" decision-making.

4.3.1. Effective of Component

Table 3 demonstrates the impact of each component. Firstly, introducing MSG3D yielded an improvement of 15.0% in SR and 7.8% in SPL (row 2). Subsequently, VVD further gains 12.5% in SR and 6.7% in SPL (row 3). Notably, introducing either AVU (row 4) or CLR (row 5) alone can only improve a part of categories. This is because AVU's new vocabulary may introduce some insufficient perception results, while the strict decision-making of CLR may hinder navigation toward detailed language-described targets. However, the optimal performance of simultaneously incorporating AVU and CLR (row 6) shows that AVU and CLR are complementary. AVU can supplement extra perception information for the strict decision-making of CLR, while CLR can address the insufficient perception of AVU. These results validate the effectiveness of each component.

4.3.2. Advantage of M3DSG

Table 4 shows the experimental results comparing MSGNav using Node-only, Concept-graph [9], and our M3DSG. Concept-graph outperforms Node-only by 4.4% in SR and 1.5% in SPL, especially for Language and Image goals, underscoring the value of relationship edges. M3DSG further boosts these categories with a total of 3.8% SR and 4.3% SPL, showing its effectiveness in scene understanding. It is worth noting that node-only performs well on category goals, mainly because such tasks rely on concise object sets rather than complex contextual information.

4.3.3. Decision-making for "Last-mile"

Table 5 validates the effectiveness of VVD in addressing the "last-mile" problem. Navigating without VVD yields only a 33.91% SR at the standard threshold of $d = 0.25$ m. As the threshold is relaxed, its SR can be significantly improved, indicating that lots of failed tasks stop near the target due to incorrect navigation point selection. Navigating with VVD can improve SR to 51.91%, which primarily recovers cases within 0.25–1 m. Fig. 5 further confirms the reliability of the visibility score estimated by the VVD: the higher-scoring viewpoint is closer to the GT viewpoint, and viewpoints with scores above 0.6 are always sufficiently close to the GT viewpoints. Nevertheless, Table 5 also shows that the gains of navigating with VVD can still be achieved as

Module				Success Rate				SPL			
M3DSG	VVD	AVU	CLR	Overall	Category	Language	Image	Overall	Category	Language	Image
				28.8	29.9	26.8	29.5	20.2	21.0	18.7	21.0
✓				43.8	56.0	35.1	40.2	28.0	31.3	23.5	29.1
✓	✓			56.3	61.6	55.0	52.3	34.7	34.9	33.9	35.2
✓	✓	✓		55.3	58.6	51.7	55.7	36.7	**35.3**	**34.8**	40.1
✓	✓		✓	53.2	**64.6**	38.5	55.7	32.9	34.8	25.8	37.7
✓	✓	✓	✓	**60.0**	63.6	**57.2**	**59.1**	**37.0**	35.0	33.4	**42.6**

Table 3. Component ablation experiment across the first episode of each scene on the "*Val Unseen*" split of GOAT-Bench. The first row without any module, which represents our baseline model 3D-Mem [43] results. "VVD", "AVU", and "CRV" represent the Visibility-based Viewpoint Decision module, Adaptive Vocabulary Update module, and Closed-loop Reasoning and Verification module.

3D Scene Graph	Success Rate				SPL			
	Overall	Category	Language	Image	Overall	Category	Language	Image
Node-only	51.8	**63.6**	44.0	47.8	31.2	**39.3**	20.5	33.8
Traditional graph [9]	56.2	**63.6**	52.8	52.3	32.7	35.3	27.3	35.5
M3DSG (Ours)	**60.0**	**63.6**	**57.2**	**59.1**	**37.0**	35.0	**33.4**	**42.6**

Table 4. Scene graph experiment across the first episode of each scene on the "*Val Unseen*" split of GOAT-Bench. "Node-only" indicates Concept-graph [9] without object relation edges. "Traditional graph" indicates Concept-graph [9].

Success Threshold	Success Rate (MSGNav)	
d (m)	w/o VVD	w/ VVD
0.25 (standard)	33.91	51.97
0.35	48.93	59.69
0.45	54.59	61.73
0.55	57.44	63.03
0.65	59.65	63.78
0.75	60.40	64.52
0.85	61.30	65.22
1.00	62.38	66.52

Table 5. Experiments on the "*Val Unseen*" split of GOAT-Bench with various success thresholds. VVD module represents the Visibility-based Viewpoint Decision module.

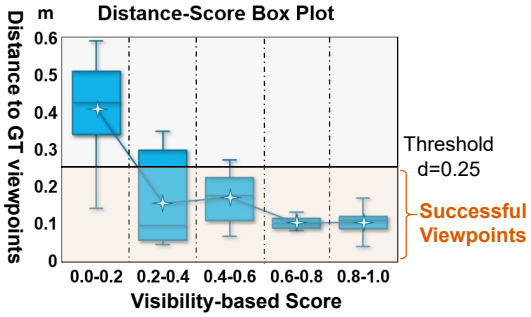


Figure 5. Statistical box plot of candidate viewpoint scores computed by the VVD module and distances from GT viewpoints.

the threshold is relaxed, demonstrating that VVD mitigates but does not eliminate the "last-mile" problem.

5. Conclusion

In this paper, we propose the MSGNav, a zero-shot embodied navigation framework built upon a Multi-modal 3D Scene Graph (M3DSG) that preserves visual information for efficient construction, robust perception, and unconstrained vocabulary. Meanwhile, we integrate a visibility-based viewpoint decision module in MSGNav to resolve the last-mile problem. Extensive experiments on GOAT-Bench and HM3D-ObjNav datasets demonstrate consistent and notable gains over existing methods, highlighting the effectiveness of the proposed multi-modal scene representations in embodied navigation.

Limitations and future work. Despite these advantages of MSGNav, scene graph-based methods still face low inference efficiency due to the latency of VFMs and VLMs, suggesting future work on faster graph construction and inference for real-time deployment. Additionally, while the "last-mile" problem has been mitigated by the VVD module, it is not fully resolved. Exploring reinforcement learning approaches, e.g. active perception, to further solve this challenge is also a way worthy of attention.

Acknowledgements. This work was supported in part by the National Natural Science Foundation of China (No. 42571514) and the Zhongguancun Academy (Grant No. 02012405).

References

[1] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. ARXIV, 2023. 2, 3

[2] Angel Chang, Angela Dai, Thomas Funkhouser, Maciej Halber, Matthias Niessner, Manolis Savva, Shuran Song, Andy Zeng, and Yinda Zhang. Matterport3d: Learning from rgb-d data in indoor environments. ARXIV, 2017. 2

[3] Haonan Chang, Kowndinya Boyalakuntla, Shiyang Lu, Siwei Cai, Eric Jing, Shreesh Keskar, Shijie Geng, Adeeb Abbas, Lifeng Zhou, Kostas Bekris, et al. Context-aware entity grounding with open-vocabulary 3d scene graphs. ARXIV, 2023. 3

[4] Jiaqi Chen, Bingqian Lin, Ran Xu, Zhenhua Chai, Xiaodan Liang, and Kwan-Yee K Wong. Mapgpt: Map-guided prompting with adaptive path planning for vision-and-language navigation. ARXIV, 2024. 3

[5] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. Yolo-world: Real-time open-vocabulary object detection. In CVPR, pages 16901–16911, 2024. 4, 5, 1

[6] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In ICCV, pages 5828–5839, 2017. 5

[7] Samir Yitzhak Gadre, Mitchell Wortsman, Gabriel Ilharco, Ludwig Schmidt, and Shuran Song. Cows on pasture: Baselines and benchmarks for language-driven zero-shot object navigation. In CVPR, pages 23171–23181, 2023. 3

[8] Dylan Goetting, Himanshu Gaurav Singh, and Antonio Loquercio. End-to-end navigation with vision language models: Transforming spatial reasoning into question-answering. ARXIV, 2024. 6, 7

[9] Qiao Gu, Ali Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, et al. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In ICRA, pages 5021–5028. IEEE, 2024. 3, 4, 7, 8

[10] Tianjun Gu, Linfeng Li, Xuhong Wang, Chenghua Gong, Jingyu Gong, Zhizhong Zhang, Yuan Xie, Lizhuang Ma, and Xin Tan. Doraemon: Decentralized ontology-aware reliable agent with enhanced memory oriented navigation. ARXIV, 2025. 7

[11] Yu He, Da Huang, Zhenyang Liu, Zixiao Gu, Qiang Sun, Guangnan Ye, and Yanwei Fu. Schr\" odinger's navigator: Imagining an ensemble of futures for zero-shot object navigation. ARXIV, 2025. 7

[12] Haodong Hong, Yanyuan Qiao, Sen Wang, Jiajun Liu, and Qi Wu. General scene adaptation for vision-and-language navigation. ARXIV, 2025. 3

[13] Yue Hu, Junzhe Wu, Ruihan Xu, Hang Liu, Avery Xi, Henry X Liu, Ram Vasudevan, and Maani Ghaffari. Imaginative world modeling with scene graphs for embodied agent navigation. ARXIV, 2025. 3

[14] Xun Huang, Hai Wu, Xin Li, Xiaoliang Fan, Chenglu Wen, and Cheng Wang. Sunshine to rainstorm: Cross-weather knowledge distillation for robust 3d object detection. In AAAI, pages 2409–2416, 2024. 3

[15] Xun Huang, Jinlong Wang, Qiming Xia, Siheng Chen, Bisheng Yang, Xin Li, Cheng Wang, and Chenglu Wen. V2x-r: Cooperative lidar-4d radar fusion with denoising diffusion for 3d object detection. In CVPR, pages 27390–27400, 2025.

[16] Xun Huang, Ziyu Xu, Hai Wu, Jinlong Wang, Qiming Xia, Yan Xia, Jonathan Li, Kyle Gao, Chenglu Wen, and Cheng Wang. L4dr: Lidar-4dradar fusion for weather-robust 3d object detection. In AAAI, pages 3806–3814, 2025. 3

[17] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. ARXIV, 2024. 2, 3

[18] Zihe Ji, Huangxuan Lin, and Yue Gao. Dynavlm: Zero-shot vision-language navigation system with dynamic viewpoints and self-refining graph memory. ARXIV, 2025. 2, 6, 7

[19] Mukul Khanna, Ram Ramrakhya, Gunjan Chhablani, Sriram Yenamandra, Theophile Gervet, Matthew Chang, Zsolt Kira, Devendra Singh Chaplot, Dhruv Batra, and Roozbeh Mottaghi. Goat-bench: A benchmark for multi-modal lifelong navigation. In CVPR, pages 16373–16383, 2024. 2, 6, 7

[20] Ue-Hwan Kim, Jin-Man Park, Taek-Jin Song, and Jong-Hwan Kim. 3-d scene graph: A sparse and semantic representation of physical environments for intelligent agents. TCYB, 50(12):4921–4933, 2019. 3

[21] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In ICCV, pages 4015–4026, 2023. 4, 1

[22] Jacob Krantz, Theophile Gervet, Karmesh Yadav, Austin Wang, Chris Paxton, Roozbeh Mottaghi, Dhruv Batra, Jitendra Malik, Stefan Lee, and Devendra Singh Chaplot. Navigating to objects specified by images. In ICCV, pages 10916–10925, 2023. 3

[23] Yuxuan Kuang, Hai Lin, and Meng Jiang. Openfmnav: Towards open-set zero-shot object navigation via vision-language foundation models. In NAACL, pages 338–351, 2024. 3

[24] Dingbang Li, Wenzhou Chen, and Xin Lin. Tina: Think, interaction, and action framework for zero-shot vision language navigation. In ICME, pages 1–6. IEEE, 2024. 3

[25] Junjie Li, Nan Zhang, Xiaoyang Qu, Kai Lu, Guokuan Li, Jiguang Wan, and Jianzong Wang. Rate-nav: Region-aware termination enhancement for zero-shot object navigation with vision-language models. ARXIV, 2025. 3

[26] LinFeng Li, Jian Zhao, Yuan Xie, Xin Tan, and Xuelong Li. Compassnav: Steering from path imitation to decision understanding in navigation. ARXIV, 2025. 7

[27] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In ECCV, pages 38–55. Springer, 2024. 5

[28] Yuxing Long, Wenzhe Cai, Hongcheng Wang, Guanqi Zhan, and Hao Dong. Instructnav: Zero-shot system for generic

instruction navigation in unexplored environment. ARXIV, 2024. 3, 7

[29] Yuxing Long, Xiaoqi Li, Wenzhe Cai, and Hao Dong. Discuss before moving: Visual language navigation via multi-expert discussions. In ICRA, pages 17380–17387. IEEE, 2024. 3

[30] Jian Luo, Jian Zhang, Jie Yang, Siwei Huang, and Bo Cai. Learning bird's eye view scene graph and knowledge-inspired policy for embodied visual navigation. KBS, page 113959, 2025. 3

[31] Dujun Nie, Xianda Guo, Yiqun Duan, Ruijun Zhang, and Long Chen. Wmnav: Integrating vision-language models into world models for object goal navigation. In IROS, pages 2392–2399. IEEE, 2025. 7

[32] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In ICML, pages 8748–8763. PmLR, 2021. 3, 4, 1

[33] Santhosh Kumar Ramakrishnan, Devendra Singh Chaplot, Ziad Al-Halah, Jitendra Malik, and Kristen Grauman. Poni: Potential functions for objectgoal navigation with interaction-free learning. In CVPR, pages 18890–18900, 2022. 3

[34] Ram Ramrakhya, Dhruv Batra, Erik Wijmans, and Abhishek Das. Pirlnav: Pretraining with imitation and rl finetuning for objectnav. In CVPR, pages 17896–17906, 2023. 2

[35] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. ARXIV, 2023. 3

[36] Tixiao Shan, Abhinav Rajvanshi, Niluthpol Mithun, and Han-Pang Chiu. Graph2nav: 3d object-relation graph generation to robot navigation. ARXIV, 2025. 2

[37] Abdelrhman Werby, Chenguang Huang, Martin Büchner, Abhinav Valada, and Wolfram Burgard. Hierarchical open-vocabulary 3d scene graphs for language-grounded robot navigation. In ICRA, 2024. 2

[38] Erik Wijmans, Abhishek Kadian, Ari Morcos, Stefan Lee, Irfan Essa, Devi Parikh, Manolis Savva, and Dhruv Batra. Dd-ppo: Learning near-perfect pointgoal navigators from 2.5 billion frames. ARXIV, 2019. 3, 2

[39] Zhenyu Wu, Angyuan Ma, Xiuwei Xu, Hang Yin, Yinan Liang, Ziwei Wang, Jiwen Lu, and Haibin Yan. Moto: A zero-shot plug-in interaction-aware navigation for general mobile manipulation. In CoRL, pages 2933–2948. PMLR, 2025. 6

[40] Zhenyu Wu, Yuheng Zhou, Xiuwei Xu, Ziwei Wang, and Haibin Yan. Momanipvla: Transferring vision-language-action models for general mobile manipulation. In CVPR, pages 1714–1723, 2025. 6

[41] Karmesh Yadav, Ram Ramrakhya, Santhosh Kumar Ramakrishnan, Theo Gervet, John Turner, Aaron Gokaslan, Noah Maestre, Angel Xuan Chang, Dhruv Batra, Manolis Savva, et al. Habitat-matterport 3d semantics dataset. In CVPR, pages 4927–4936, 2023. 2, 5, 6, 7

[42] Zichen Yan, Rui Huang, Lei He, Shao Guo, and Lin Zhao. Sign: Safety-aware image-goal navigation for autonomous drones via reinforcement learning. ARXIV, 2025. 3

[43] Yuncong Yang, Han Yang, Jiachen Zhou, Peihao Chen, Hongxin Zhang, Yilun Du, and Chuang Gan. 3d-mem: 3d scene memory for embodied exploration and reasoning. In CVPR, pages 17294–17303, 2025. 2, 3, 5, 6, 7, 8

[44] Hang Yin, Xiuwei Xu, Zhenyu Wu, Jie Zhou, and Jiwen Lu. Sg-nav: Online 3d scene graph prompting for llm-based zero-shot object navigation. NeurIPS, 37:5285–5307, 2024. 2, 3, 4, 6, 7

[45] Hang Yin, Xiuwei Xu, Linqing Zhao, Ziwei Wang, Jie Zhou, and Jiwen Lu. Unigoal: Towards universal zero-shot goal-oriented navigation. In CVPR, pages 19057–19066, 2025. 2, 3, 4

[46] Naoki Yokoyama, Sehoon Ha, Dhruv Batra, Jiuguang Wang, and Bernadette Bucher. vlfm: Vision-language frontier maps for zero-shot semantic navigation. In ICRA, pages 42–48. IEEE, 2024. 3, 7, 2

[47] Naoki Yokoyama, Ram Ramrakhya, Abhishek Das, Dhruv Batra, and Sehoon Ha. Hm3d-ovon: A dataset and benchmark for open-vocabulary object goal navigation. In IROS, pages 5543–5550. IEEE, 2024. 2, 6

[48] Bangguo Yu, Hamidreza Kasaei, and Ming Cao. L3mvn: Leveraging large language models for visual target navigation. In IROS, pages 3554–3560. IEEE, 2023. 7

[49] Zhaohuan Zhan, Lisha Yu, Sijie Yu, and Guang Tan. Mc-gpt: Empowering vision-and-language navigation with memory map and reasoning chains. ARXIV, 2024. 3

[50] Jiazhao Zhang, Kunyu Wang, Shaoan Wang, Minghan Li, Haoran Liu, Songlin Wei, Zhongyuan Wang, Zhizheng Zhang, and He Wang. Uni-navid: A video-based vision-language-action model for unifying embodied navigation tasks. ARXIV, 2024. 2

[51] Kaiwen Zhou, Kaizhi Zheng, Connor Pryor, Yilin Shen, Hongxia Jin, Lise Getoor, and Xin Eric Wang. Esc: Exploration with soft commonsense constraints for zero-shot object navigation. In ICML, pages 42829–42842. PMLR, 2023. 3

[52] Ziyu Zhu, Xilin Wang, Yixuan Li, Zhuofan Zhang, Xiaojian Ma, Yixin Chen, Baoxiong Jia, Wei Liang, Qian Yu, Zhidong Deng, et al. Move to understand a 3d scene: Bridging visual grounding and exploration for efficient and versatile embodied navigation. ARXIV, 2025. 2, 7

[53] Filippo Ziliotto, Tommaso Campari, Luciano Serafini, and Lamberto Ballan. Tango: training-free embodied ai agents for open-world tasks. In CVPR, pages 24603–24613, 2025. 2, 7

MSGNav: Unleashing the Power of Multi-modal 3D Scene Graph for Zero-Shot Embodied Navigation

Supplementary Material

6. Implement Details

This section outlines the implementation details of our approach, including computational resource, hyper-parameters, and the vision foundation models (VFMs) used for constructing M3DSG.

6.1. Computational Resource

Our approach primarily employs a Vision-Language Model (VLM) for inference. Consequently, when utilizing API calls to closed-source models, it places only modest demands on local GPU computational resources. All experiments were conducted using NVIDIA A800-SXM4-40GB graphics cards, with each card requiring approximately 8 GB to 10 GB of VRAM to perform VFMs inference. This configuration ensures stable performance while maintaining cost-effectiveness in resource usage.

Meanwhile, inference with open-source large models such as GPT-OSS [2] and LLAVA [3] is also feasible. However, this configuration demands additional GPU memory that scales in proportion to the parameter size of open-source large models.

6.2. Hyper-parameters

The details of the hyperparameters of `MSGNav` are shown in Table 6. Additionally, we maintain consistency with 3D-Mem and Conceptgraph for other parameters in the 3D scene graph construction.

Hyperparameters	Value
Radii set: $\mathbf{R}$	[0.5, 0.75] (m)
Camera height: h,	1.5 (m)
Success threshold: d	0.25 (m)
Samples number: K	20
Adjacency distance: θ	3.5 (m)
Obstruction distance: τ	0.05 (m)
Top-k related objects: $\lvert\mathbf{O}^{rel}\rvert$	20
Hfov of camera	120°
Image resolution	1280x1280
Image prompt resolution	512x512

Table 6. The details of hyperparameters of `MSGNav`.

6.3. VFMs for Constructing M3DSG

As outlined in the main paper, the 3D scene reconstruction pipeline primarily incorporates the following Visual Feature Modules (VFMs). First, YOLO-World [5] is applied for object detection and room recognition. Next, SAM [21] is utilized to generate semantic masks. Finally, CLIP [32] is employed to extract visual feature representations of the objects.

[2]GPT-OSS: https://github.com/openai/gpt-oss
[3]LLAVA: https://github.com/haotian-liu/LLaVA



Figure 6. A visualization example of perception results during constructing the M3DSG.

An example of visualized perception results is shown as Fig. 6, illustrating the outputs obtained after processing with YOLO-World and SAM for open-vocabulary object detection and semantic segmentation.

And the detailed information of VFMs is shown in Table 7.

VFM	Version	Size
Yolo-World [5]	YOLOv8x-world	141.11 MB
SAM [21]	SAM-Large	1.16 GB
CLIP [32]	ViT-H-14-quickgelu/dfn5b	12.20 GB

Table 7. The detailed information of VFMs for constructing M3DSG.

6.4. Illustration of End-to-end Framework

In addition to the framework diagram in the main paper, we also provide a more intuitive end-to-end framework diagram that systematically illustrates the data flow and processing logic between modules, as shown in Fig. 7.

The system operates in a continuous sense-reason-act loop at each timestep t, which can be detailed in four key stages: **(1) Constructing Multi-modal 3D Scene Graph:** Given the current observation $\mathcal{I}_t$ and agent pose, Vision Foundation Models (VFMs)

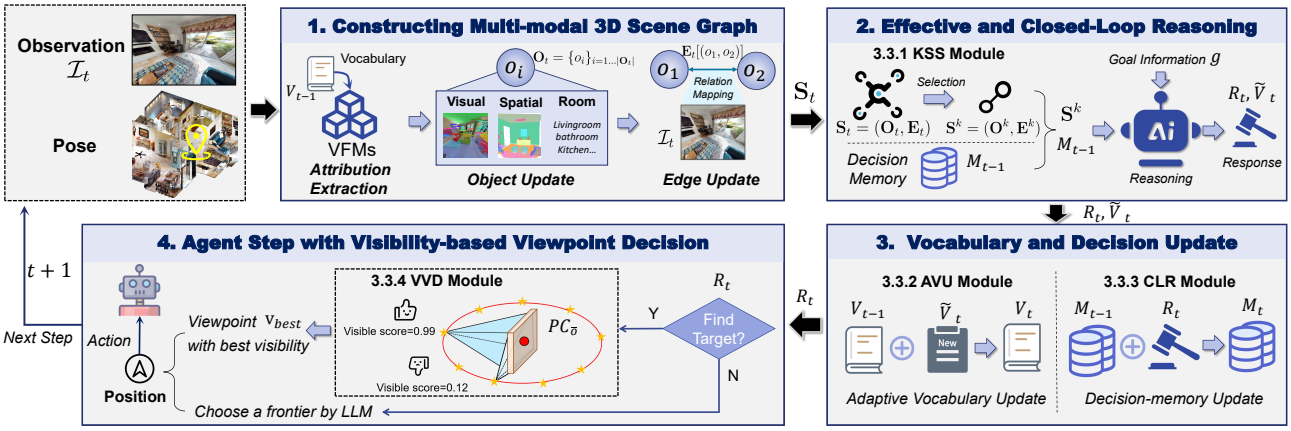


Figure 7. **Detailed end-to-end processing flow of the proposed framework.** The system executes a continuous sense-reason-act loop across four key stages: (1) multi-modal 3D scene graph construction via Vision Foundation Models (VFMs), (2) closed-loop reasoning based on the extracted sub-graph $\mathbf{S}^k$ and historical memory, (3) adaptive updates to the semantic vocabulary and decision memory, and (4) visibility-based viewpoint decision or frontier exploration. Please refer to the corresponding text for a detailed description of each mathematical symbol and data flow.

Method	Training-free	SR ↑	SPL ↑
BCRL [34]	×	20.2	8.2
DAgger [34]	×	18.1	9.4
DAgRL [34]	×	41.3	21.2
RL [38]	×	39.2	18.7
VLFM [46]	✓	35.2	18.6
Uni-NaVid [50]	×	41.3	21.2
DAgRL+OD [47]	×	35.8	21.2
MTU3D [52]	×	**55.0**	23.6
MSGNav (Ours)	✓	48.3	**27.0**

Table 8. Comparison of navigation performance on the **Val Seen** split of HM3D-OVON. This experiment evaluates the model's optimization effectiveness and navigation efficiency within the training distribution.

extract visual, spatial, and room-level attributes to incrementally update the object nodes and relational edges of the scene graph $\mathbf{S}_t$. **(2) Effective and Closed-Loop Reasoning:** A Key Sub-graph Selection (KSS) module filters $\mathbf{S}_t$ into a task-relevant sub-graph $\mathbf{S}^k$. An LLM agent then reasons over $\mathbf{S}^k$, historical decision memory M_{t-1}, and goal information $\mathcal{G}$ to generate an action response R_t and proposed vocabulary updates $\tilde{V}_t$. **(3) Vocabulary and Decision Update:** The Adaptive Vocabulary Update (AVU) module integrates $\tilde{V}_t$ to form the current vocabulary V_t, while the Closed-Loop Reasoning (CLR) module records R_t to update the decision memory M_t. **(4) Agent Step with Visibility-based Viewpoint Decision:** If the target is identified, the VVD module selects the optimal viewpoint $\mathbf{v}_{best}$ that maximizes the visibility score of the target point cloud $PC_{\bar{o}}$; otherwise, the LLM selects a new frontier for exploration in step $t+1$.

7. More Experimental Results

7.1. Additional Analysis on HM3D-OVON

To further investigate the foundational perception and optimization stability of the proposed MSGNav, we conduct supplementary experiments on the HM3D-OVON dataset [47]. For this analysis, we specifically report the results on the "*Val Seen*" split to evaluate the model's performance when navigating within familiar environment distributions, which serves as a benchmark for the model's upper-bound efficiency and basic semantic mapping capabilities.

As presented in Table 8, we compare MSGNav with several state-of-the-art training-based and training-free methods on the seen environments. While the training-based method MTU3D [52] achieves a higher Success Rate (SR) of 55.0% by leveraging extensive environment-specific learning, MSGNav significantly outperforms all methods in terms of Success weighted by Path Length (SPL), reaching **27.0%**.

This result is particularly noteworthy as MSGNav is a *training-free* approach. The superior SPL indicates that even in seen scenarios, our method generates more efficient and purposeful navigation paths compared to models that might overfit to specific trajectories. These supplementary results confirm that MSGNav maintains robust basic navigation logic and high path efficiency, which provides a solid foundation for the zero-shot generalization capabilities demonstrated in our primary experiments (Table 1 in the main paper).

7.2. Quantitative experiments on Goat-Bench

We present the detailed experimental results in Table 2 of the main paper, showing the detailed performance of MSGNav across different categories with various VLM. As shown in the table 9, MSGNav demonstrates strong adaptability across different VLMs instead of relying on the specific model. MSGNav achieves comparable results on both state-of-the-art VLMs, Qwen-VL-Max and GPT-4o. It is worth noting that the results on Qwen-VL-Max are

VLM	**Overall** (2669)		**Object Category** (991)		**Language** (856)		**Image** (822)	
(MSGNav)	Success Rate	SPL	Success Rate	SPL	Success Rate	SPL	Success Rate	SPL
GPT-4o [17]	51.97	29.56	53.38	29.98	**47.43**	25.08	54.99	**33.71**
Qwen-VL-Max [1]	**52.79**	**30.79**	**56.10**	**31.53**	45.56	**27.18**	**56.33**	33.67

Table 9. Experiments of our MSGNav method using different VLMs on the "*Val Unseen*" split of GOAT-Bench. The number following each category represents the sample size.

slightly better than those on GPT-4o. This is primarily due to the costly token fees of GPT-4o, our prompts were primarily implemented based on Qwen-VL-Max, and they may be better suited for Qwen-VL-Max. However, the excellent results achieved on GPT-4o without any prompt adjustments demonstrate that the system does not overly rely on prompt engineering.

8. Analysis of the Exploration

8.1. Additional Details of VVD Module

We provide an example illustrating how the VVD module selects optimal viewpoints. As shown in Fig. 8, candidate viewpoints located closer to the ground-truth (GT) viewpoints receive higher visibility scores. In this example, the highest-scoring viewpoint (Viewpoint 1, score 0.97) is positioned directly in front of the target, offering an unobstructed field of view that clearly captures the target object.

By contrast, the medium-scoring viewpoint (Viewpoint 2, score 0.83) is positioned on the side of the target, thereby capturing only partial visual information. Notably, Viewpoint 3 (score 0.05) is severely occluded by environmental structures, which may be less discernible purely from a simplified 2D bird's-eye view (BEV) representation. In such cases, the VVD module correctly identifies the occlusion by computing 3D line-of-sight, consistently assigning low visibility scores to these obstructed viewpoints.

To further clarify the mechanics of the VVD module, we expand upon the operations detailed in Algorithm 2 of the main paper. The algorithm systematically evaluates line-of-sight occlusion in 3D space to identify the optimal navigation endpoint.

1. **Candidate Generation (Lines 1-4):** The algorithm first computes the centroid $\mathbf{c}_{\bar{o}}$ of the localized target's point cloud $\mathcal{PC}_{\bar{o}}$. It then generates a set of K candidate viewpoints in a concentric circle (or multiple concentric circles defined by radii set $\mathbf{R}$) around this centroid. Crucially, it filters this set to retain only viewpoints $\mathbf{V}_c$ that reside in traversable free space, ensuring the agent can physically reach them.
2. **Ray Construction (Line 7):** For a given candidate viewpoint $\mathbf{v}_i$ and a specific point $\mathbf{p}$ belonging to the target object, the algorithm mathematically defines a ray segment $\mathcal{Q}(\mathbf{v}_i, \mathbf{p})$. This segment connects the viewpoint and the target point. To account for the physical volume of the agent and potential sensor noise, the parameter τ introduces a safety margin at both ends of the ray.
3. **Occlusion Evaluation (Line 8):** The core visibility condition $\mathcal{E}(\mathbf{v}_i, \mathbf{p})$ evaluates whether the defined ray $\mathcal{Q}$ is obstructed by the broader scene point cloud $\mathcal{PC}$. A target point $\mathbf{p}$ is considered visible if and only if the minimum distance between any point $\mathbf{q}$ on the ray and any point $\mathbf{s}$ in the environment point cloud is strictly greater than or equal to the obstruction margin τ.
4. **Visibility Scoring (Lines 9-13):** The algorithm computes an aggregate visibility score $S_{\mathbf{v}_i}$ for the candidate viewpoint. This score represents the ratio of target points $\mathbf{p} \in \mathcal{PC}_{\bar{o}}$ that satisfy the visibility condition $\mathcal{E}$. The viewpoint achieving the highest ratio is selected as the optimal navigation target $\mathbf{v}_{best}$, guaranteeing the agent finishes its trajectory with a robust, unoccluded view of the target.

8.2. Visualization about VVD Module

We provide a real-world example illustrating how the VVD module selects optimal viewpoints. As shown in Fig. 9, candidate viewpoints located closer to the ground-truth (GT) viewpoints, marked by the purple triangle, receive higher visibility scores. In this example, the highest-scoring viewpoint is almost coincident with the GT viewpoint and is positioned directly in front of the target (top), offering an unobstructed field of view.

By contrast, the previous viewpoint selection was farther from the GT viewpoint and positioned on the side of the target, thereby capturing only partial visual information. Notably, no GT viewpoint exists at the rear (lower side) of the target due to occlusion caused by point cloud data (green dots) at the same height within the scene, which may be less discernible in the bird's-eye view (BEV) representation. In such cases, the VVD module consistently assigns low visibility scores (0.1–0.6) to these occluded viewpoints.

8.3. Visualization about Exploration

As shown in Fig. 10, we present the natural language-guided target navigation by visualizing the agent's process of exploring and localizing the target described as "refrigerator in the kitchen. It is located next to the kitchen cabinet and the worktop" to intuitively demonstrate its closed-loop reasoning.

The agent starts at the predefined Start Position and begins to explore adjacent areas. At this stage, the target refrigerator is still unseen (indicated by a red dot). The agent continuously moves along the Frontier (the boundary between explored and unexplored areas, marked by purple arrows) to uncover more regions. Its movement trajectory is recorded as the Exploration path (light gray line), and its Current Position (blue dot) is updated dynamically as it navigates through different rooms. As the agent ventures into the kitchen area, the Observation sequence captures the visual input of the refrigerator adjacent to the kitchen cabinet and worktop. Simultaneously, in the Top-down Map, the target marker changes from a red dot (Target (unseen)) to a green dot (Target (seen)), and the agent reaches the Final Position (green triangle), completing this hard navigation task.

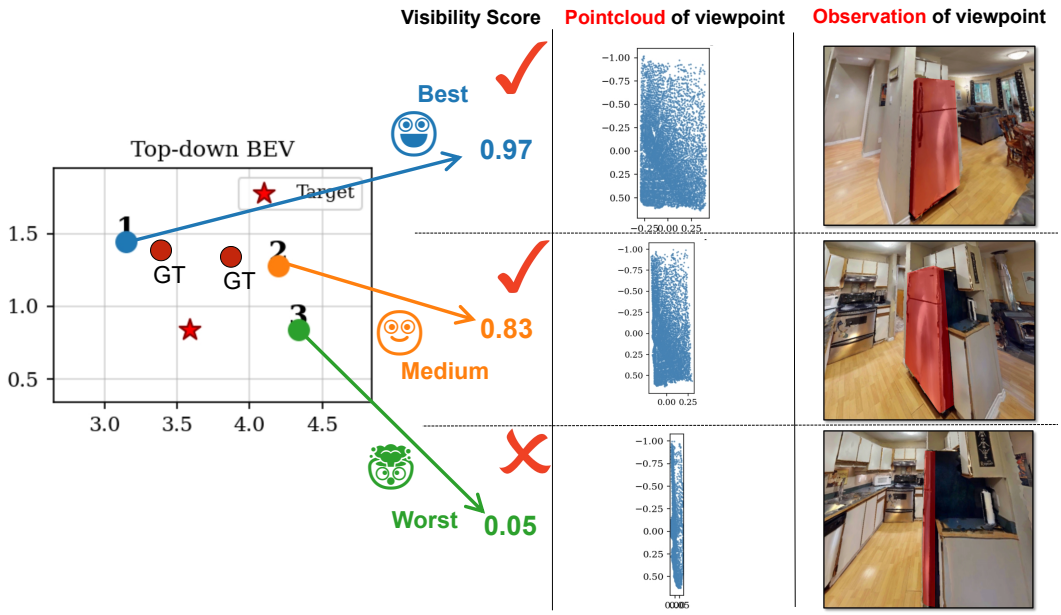


Figure 8. **Qualitative Visualization of the Visibility-based Viewpoint Decision (VVD) Module.** The central plot shows a top-down Bird's-Eye View (BEV) of the agent's spatial environment. Red stars indicate potential targets, while red circles mark the Ground Truth (GT) target viewpoints. The VVD module evaluates candidate viewpoints (numbered 1, 2, and 3), calculating a visibility score based on ray-casting to the target point cloud. Viewpoint 1 achieves the 'Best' visibility score (0.97), and its corresponding first-person observation provides a clear, unoccluded view of the red refrigerator. Viewpoint 2 receives a 'Medium' score (0.83), capturing a partial, side-angle view. Viewpoint 3 receives the 'Worst' score (0.05), as its line of sight is heavily occluded by other environmental structures.

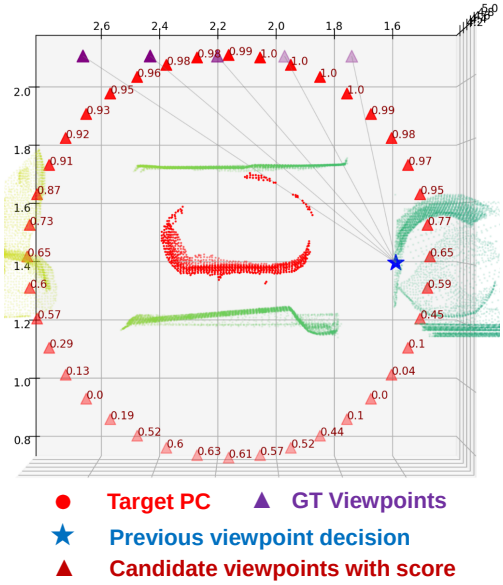


Figure 9. An example of viewpoint decision by the VVD module. This shows the BEV view of the point cloud in the scene and the corresponding viewpoint information.

This visualization comprehensively demonstrates how the agent uses the multi-modal scene graph to sequentially reason and localize the target object in an unknown environment. With the help of our multi-modal scene graph, agents can gradually explore and locate targets even when they are extremely distant. This further validates the effectiveness of our multi-modal scene graph for reasoning in zero-shot embodied navigation tasks.

Furthermore, to explicitly illustrate the critical role of our proposed M3DSG in the granular decision-making process, we visualize the Vision-Language Model (VLM) reasoning process in Fig. 11. In this episode, the agent is tasked with finding a target based on a complex spatial description: "herber carpet is the room. It is located near the rack, freezer, and refrigerator towards the bottom right corner of the frame". As the exploration proceeds ($t = 1 \dots T$), the M3DSG incrementally updates. When matching the target, the VLM relies on the M3DSG Nodes to identify the candidate, explicitly noting that the described object matches Object 13. More importantly, it heavily depends on the Multi-modal Edges (stored images) to verify fine-grained spatial relations. The reasoning process highlights that specific images (particularly Image 1, 2, and 5) show a large red carpet positioned near a refrigerator (Object 11), a shelf, and a freezer. Additionally, the visual edges confirm the carpet's placement near the bottom right corner of the frame, perfectly aligning with the language description. By combining node attributes and visual edge evidence, the agent confidently concludes that Object 13 is the correct match. Ultimately, this accurate multimodal reasoning allows the agent to efficiently succeed in the task, reaching the target viewpoint at a remarkably close distance of 0.127m.



Figure 10. This visualization illustrates the process of an agent exploring and navigating to a target described by natural language—"refrigerator in the kitchen. It is located next to the kitchen cabinet and the worktop"—in an indoor environment. It is composed of three parts from top to bottom: the Top-down Map sequence, the Observation sequence (first-person visual inputs), and the legend, which respectively display the global exploration state, the agent's local visual perception, and the meaning of each visual element.

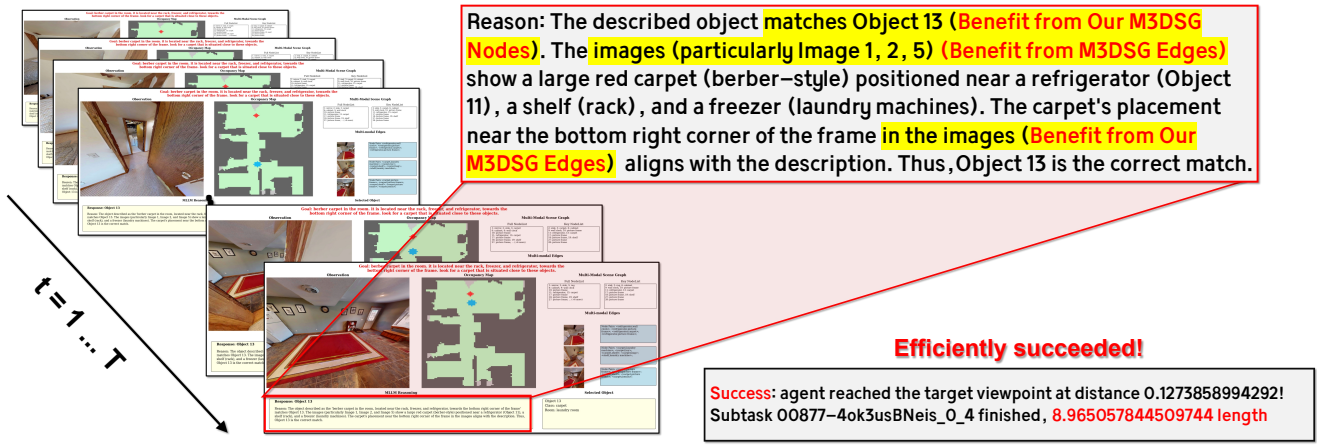


Figure 11. **Visualization of the VLM reasoning process based on M3DSG.** The agent successfully locates the target "berber carpet" by leveraging M3DSG Nodes to match the object candidates (Object 13) and Multi-modal Edges (images) to verify complex spatial relationships, such as the carpet's placement near the refrigerator (Object 11) and its position in the bottom right corner of the frame. This accurate grounding leads to an efficiently successful navigation task.

8.4. Analysis of Reasoning based on M3DSG

We present an example illustrating the use of a Vision-Language Model (VLM) for multimodal scene graph reasoning. As shown in Fig. 12, when given the query "plant that is located near the window glass and radiator", the conventional textual description in the scene graph failed to include information pertinent to the target. In contrast, the VLM, utilizing the supplementary visual information embedded in the scene graph, successfully identified the target matching the query. The model then enriched the scene graph through vocabulary supplementation and updates. As a result, the task was completed with a Success weighted by Path Length (SPL) of 89.79, requiring only about 3 m of exploration length. This demonstrates that retaining images within the multi-modal scene graph substantially improves exploration efficiency.

9. VLM Reasoning and Prompt

Our **MGSNav** method leverages Vision-Language Model (VLM) reasoning in three scenarios during the navigation process:

- **Top-k object nodes selection in KSS** (As shown in Fig. 13) — Within the Key Scene Selection (KSS) module, the VLM is applied to infer the top-k target-related nodes from the compressed scene graph.
- **Exploration inference** — The KSS-compressed key scene subgraph is processed by the VLM to identify either the target node or the exploration frontier. This reasoning is performed in two sequential steps:
  1. **Find target**. If the target is present in the 3D scene graph, input the scene graph information to identify the target ID. (As shown in Fig. 14)
  2. **Explore frontier**. If absent, input the frontier image to determine the exploration frontier image ID. (As shown in Fig.

15)

This decomposition facilitates more accurate VLM reasoning.

- **Task completion verification** — Once the agent considers the task completed, the VLM utilizes observation images from the past steps to perform a final inference, thereby validating task credibility. (As shown in Fig. 16)

To better understand the process of VLM reasoning, we present the prompts used for the three VLM reasoning scenarios described above.

1. Question?

Could you find the object exactly described as the 'plant that is located near the window glass and radiator.'?

2. Input Multi-modal Scene Graph

Objects Attribution:

radiator: 8, (10.99, -2.15, 0.36), kitchen room
chair: 10, (11.30, -1.26, 0.87), kitchen room
bed: 11, (10.85, -0.64, 0.64), kitchen room
chair: 12, (11.33, -0.10, 0.49), kitchen room
chair: 18, (10.74, -1.28, 0.87), kitchen room

Relationship Attribution:

8, 8, [Image 0, Image 1]
8, 10, [Image 0]
8, 11, [Image 0, Image 1]
8, 12, [Image 0]
8, 18, [Image 1]

Image List:

Image 0 [iVBORw0KGg...]
Image 1 [iVBORw0KGg...]

3. Response and Reason from VLM

Response: [image 0, plant]

Reason: [The image shows a plant located near the window and radiator, matching the description.]

4. Task Complete

- Success: agent reached the target viewpoint at distance 0.1468908041715622!
- Subtask 00890-6s7QHgap2fW_1_0 finished, 3.2773782577578823 length
- SPL by distance: 89.79
- Filtered frames/Total frames: 2/26

Figure 12. Example illustrating the use of a Vision-Language Model (VLM) for multimodal scene graph reasoning.

System Prompt:

You are an agent in an indoor scene who can observe the environment and explore to find a target object. You must choose an Image or an Object as the answer, in order to find the specified target object.

To efficiently solve the problem, you should identify key objects that are most helpful for guiding exploration toward the target. Please follow these strict instructions:
1. Read and understand the full 3D scene graph. Each object includes its id, class, room, and nearby objects (i.e., its neighbors in the graph).
2. Rank objects by how helpful they are for locating the target, based on: Semantic relevance to the target; Co-occurrence with the target in typical environments; Presence in the same room as the target.
3. Choose only the most informative and strategically diverse objects for exploration. To maximize coverage: Avoid choosing objects that are directly connected (i.e., neighbors) in the scene graph.Here is is the format for input 3D scene graph:
Object ID: Class, Located room, nearby objects ID
Question: {*Example Question*}.
{*Example Scene Graph*}
Answer:
1
5

Content Prompt:

Following is the concrete content of the task and you should retrieve helpful key objects in order.
Question: {*Task Question*}
Following is the 3D scene graph based on the above input format
1: spa bench, laundry room, [1, 2]
2: wall cabinet, laundry room, [1, 2]
{. . .}
Do not print any object that are not included in the 3D scene graph or include any additional information other than the ID in your response:
Answer:

Figure 13. Prompt 1 for "top-k object nodes selection in KSS module". The placeholders {...} will be replaced by the corresponding information. The gray-highlighted text represents the information of compressed M3DSG.

System Prompt:

```
You are an agent in an indoor scene who can observe the environment and explore
to find a target object. You must choose an Image, a Frontier (for further
exploration), or directly select an Object as the answer, in order to find the
specified target object.
Scene Graph Definitions:
    Objects Attribution: Each object's name, ID, position, and location.
    Relationship Attribution: Relationships between objects, often with a
reference image.
    Image List: Supplementary images from the nodes of scenegraph.
    Frontier: Unexplored regions that may provide new information.
    History Decision: All previous choices you made (avoid repeating them).
Instructions:
    Step 1: Examine the Objects Attribution section. If the target object is
explicitly listed (by name or ID), and fits the question, select it immediately
as the answer.
    Step 2: If the target object is not explicitly in Objects Attribution, check
the Relationship Attribution section. Use relationships and referenced images in
the Image List to help you identify the target.
    Step 3: If you still cannot identify the object, select to further explore
and gather more information.
    Step 4: Provide your answer in one of the following formats:
        'Object i': If the object is found in Objects Attribution.
        'Image i, j': If the object is likely to exist in image i, and j is the
required object category. Ensure you include the category name.
        'Continue Exploration': If the object is not found and further
exploration is necessary.
Additional Notes:
1. Try not to select any object, image that is in the "History Decision" list
unless you are very confident.
2. The detected class name and located room in the Objects Attribution may be
inaccurate, use the images to verify.
3. Only provide the required answer(with optional brief reasoning in a new line).
```

Content Prompt:

Here is the Question you need to solve: {*Question*}'?
Objects Attribution:
    ventilation hood: 16, (1.99, 1.55, 1.63), kitchen room
    wall cabinet: 19, (3.17, 1.97, 0.46), kitchen room
    {...}
Relationship Attribution:
    16, 7, [Image 0]
    16, 16, [Image 0, Image 1]
    {...}
Image List:
Please note that the class name and Located room may not be accurate due to the limitation of the detection model. So you still need to utilize the images to make the decision.
    Image 0 [iVBORw0KGg...]
    Image 1 [iVBORw0KGg...]
    {...}
The followings are all the previous Decisions that you made: (now step is 10/50). Choosing those incorrect objects or images again is prohibited:
   step 0 : Choosing a Frontier to explore.
   step 1 : Choosing a Frontier to explore.
    {...}
Answer:
You can explain the reason for your choice, but put it in a new line after the choice.

Figure 14. Prompt 2 for "*find target in exploration inference*". The placeholders {...} will be replaced by the corresponding information. The gray-highlighted text represents the information of the full M3DSG.

System Prompt:

```
You are an agent tasked with finding a target object in an indoor
scene. Your mission is to choose the most promising frontier for
further exploration to locate the specified target object.
```

Content Prompt:

```
Here is the Question you need to solve: {Question}
The Frontiers that you can explore:
    Frontier 0 [iVBORw0KGg...]
    Frontier 1 [iVBORw0KGg...]
    {...}
You can explain the reason for your choice, but put it in a new
line after the choice.
The example for the answer:
Frontier 0
I chose Frontier 0 for exploration, which show direction to
kitchens where refrigerators are more likely to appear
```

Figure 15. Prompt 3 for "*explore frontier in exploration inference*". The placeholders {...} will be replaced by the corresponding information. The gray-highlighted text represents the information of frontier images as in 3D-Mem [43].

System Prompt:

```
Task: You are an agent in an indoor scene that is able to observe the
surroundings and explore the environment. You are tasked with indoor
navigation, and you are required to choose a Image or a Frontier to
explore, or directly select an Object, finally find the target object
required in the question.

Definitions:
Now that you have arrived near the previously selected answer, please
observe your surroundings and confirm whether you have really reached
the object required by the Question (close enough, less than 0.25m).
```

Content Prompt:

```
Here is the Question you need to solve: {Question}'?
The following are surrounding observations about the egocentric view
of the agent:
    Surrounding observation 1 [iVBORw0KGg...]
    Surrounding observation 2 [iVBORw0KGg...]
    {...}
Answer: (Yes or No)
You can explain the reason for your choice, but put it in a new line
after the choice.
```

Figure 16. Prompt 4 for "*task completion verification*". The placeholders {...} will be replaced by the corresponding information. The gray-highlighted text represents the information of observation images from the past steps.